

Current State Analysis

Current Infrastructure

Fremont Rocket Company manages all operational data across 6 disconnected spreadsheets with no database infrastructure

Source Files

- Sales_Data.csv
- Production_Line_Invoice_Details.xlsx
- Energy_Prices_Daily.xlsx
- Fremont_Rocket_Company_Employees.xlsx
- Sales_People.csv
- Customer_Data.csv

Why This is a Problem

- No relationships enforced between files
- No single source of truth
- Data duplicated across files
- Leadership cannot get reliable profitability numbers
- Company plans to scale, spreadsheets will not support growth

Solution – Migrate to a relational database with a schema

Bus Matrix - Data to Design

Dimension	fact_sales	fact_production_expense
dim_date	✓	✓
dim_rocket	✓	✓
dim_customer	✓	
dim_salesperson	✓	
dim_service_contract	✓	
dim_employee		✓
dim_vendor		✓
dim_description		✓

2 Facts.

2 Conformed Dimensions.

1 Unified Schema.

How we got the data model

- Identified two business (facts) processes, Sales and Production
- Built a Bus Matrix mapped dimensions to each process
- Conformed dimensions emerged dim_date and dim_rocket shared by both facts
- Schema followed naturally

Database Schema Design



Schema Architecture

The logical data model employs a Star Schema architecture designed for high-performance analytics at Fremont Rocket Co.

Why this Model

- Single source of truth
- No data duplication
- Referential integrity enforced
- Easy to query and analyze
- Plug and play scalability
- Consistent and reliable data
- Future proof, new processes plug into existing dimensions

Example Queries

Answering real business questions 1. Which rocket models are most profitable?

2. Which customers are buying the most? 3. sales of rocket year by year?

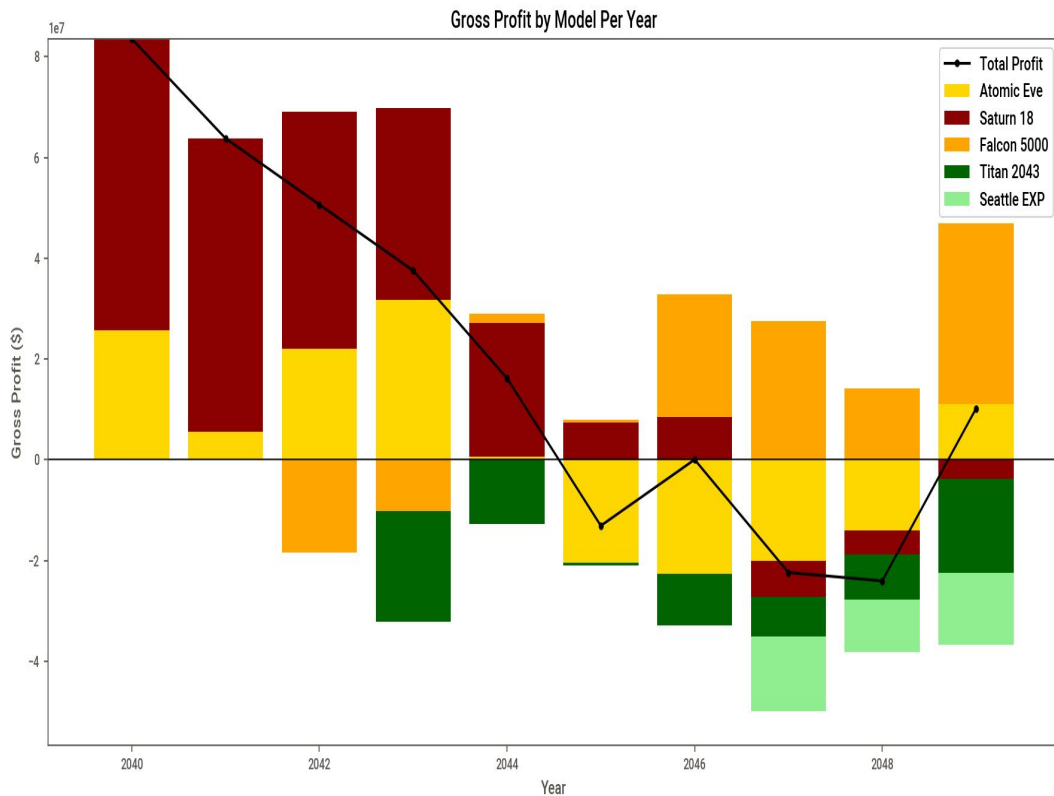
```
1. WITH yearly_revenue AS (  
    SELECT s.rocket_id, d.year,  
        COUNT(s.sale_id) AS units_sold,  
        SUM(s.sale_price) AS total_revenue,  
        SUM(s.commission_paid) AS total_commission  
    FROM fact_sales s  
    JOIN dim_date d ON s.date_id = d.date_id  
    GROUP BY s.rocket_id, d.year  
)  
yearly_costs AS (  
    SELECT p.rocket_id, d.year,  
        SUM(p.amount) AS total_production_cost  
    FROM fact_production_expense p  
    JOIN dim_date d ON p.date_id = d.date_id  
    GROUP BY p.rocket_id, d.year  
)  
SELECT r.rocket_name, rev.year,  
    rev.total_revenue, yc.total_production_cost,  
    rev.total_revenue - rev.total_commission -  
    yc.total_production_cost AS gross_profit,  
    ROUND((rev.total_revenue - rev.total_commission -  
    yc.total_production_cost)  
    / rev.total_revenue * 100, 2) AS margin_pct  
FROM yearly_revenue rev  
JOIN yearly_costs yc  
ON rev.rocket_id = yc.rocket_id  
AND rev.year = yc.year  
JOIN dim_rocket r ON rev.rocket_id = r.rocket_id  
ORDER BY rev.year, gross_profit DESC
```

```
2. SELECT  
    c.organization AS customer,  
    COUNT(fs.sale_id) AS rockets_bought,  
    SUM(fs.sale_price) AS total_spent,  
    ROUND  
    ( SUM(fs.sale_price) * 100.0 / SUM(SUM(fs.sale_price)) OVER(), 2)  
    AS pct_of_total_revenue FROM fact_sales fs JOIN  
    dim_customer c ON  
    fs.customer_id = c.customer_id  
GROUP BY c.organization  
ORDER BY total_spent DESC
```

```
3. SELECT  
    d.year, r.rocket_name,  
    COUNT(fs.sale_id) AS units_sold,  
    SUM(fs.sale_price) AS total_revenue  
FROM fact_sales fs  
JOIN dim_date d ON  
    fs.date_id = d.date_id JOIN  
    dim_rocket r ON  
    fs.rocket_id = r.rocket_id  
GROUP BY d.year, r.rocket_name  
ORDER BY d.year, total_revenue DESC
```

Profitability Analysis

Yearly Model profits contribution to total profits



Observation

- Company profitable 2040-2043 (**Saturn 18** drove the profits)
- Total profit crossed below zero in 2044
- **Titan 2043** consistently dragging profit down
- **Falcon 5000** only growing positive contributor
- Slight recovery in 2049 as Falcon 5000 takes over

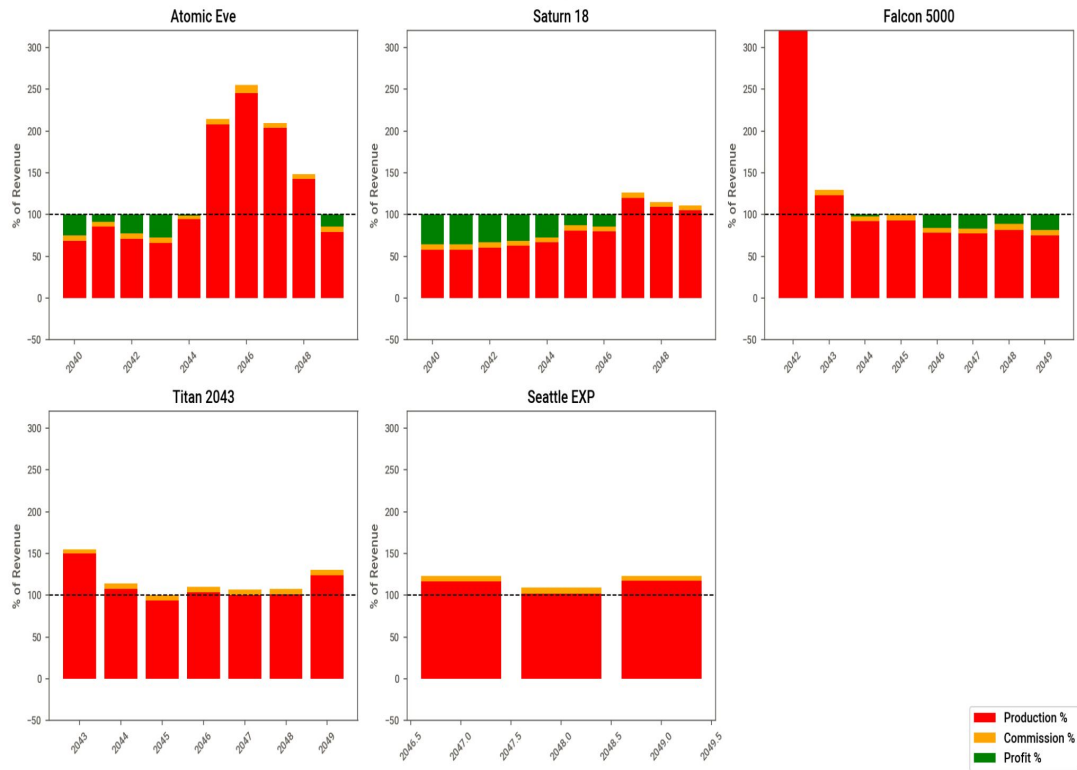
Recommendation

- Monitor Atomic Eve recovery (positive in 2049 but fragile)
- Urgently review Titan 2043 never profitable
- **Accelerate Falcon 5000** investment now

Where Does Revenue Go?

Production and commission contribution to total profits for each Rockets

Revenue Breakdown by Rocket Model



Observation

- Commission is consistently 6%
- Production cost is the real issue across all models
- **Falcon 5000** improved from 300% cost ratio to 75%
- **Titan 2043** never below 100% production cost
- **Atomic Eve** costs spiraled to 250% while sales collapsed

Recommendation

- Investigate cost price on Seattle EXP and Titan 2043: does not justify production cost
- Need better vendor contracts (production cost is primary margin killer)
- Use Falcon 5000 blueprint for other models based on cost

Data Infrastructure Landscape

Evaluating options for now and future scale

RECOMMENDED NOW



Azure SQL + Azure Data Factory

Pros:

- Quick setup in days
- Low cost at current scale
- ADF automates Excel
- Easy migration to Synapse

Cons:

- Vertical scaling only
- Limited large analytics

RECOMMENDED FUTURE



Synapse + Databricks + ADLS

Pros:

- Event-driven pipeline
- Massive scaling with Databricks
- Optimized for analytics
- Native Azure ecosystem

Cons:

- Complex/Expensive setup
- High skill set required

ALTERNATIVE FUTURE



Snowflake on Azure

Pros:

- Zero maintenance
- Auto-scales instantly
- Best-in-class query performance
- Multi-cloud flexibility

Cons:

- Outside Azure ecosystem
- Unpredictable query cost

Start with Azure SQL and ADF today. Scale to Synapse and Databricks as data volume and team grows. Snowflake is alternative if multi-cloud flexibility matters and concurrent data analytics by BI team

ETL Pipeline Design

Recommended ETL Workflow and Validation checks



ETL Steps

EXTRACT

- Sales team saves updated Excel to SharePoint
- Event Grid detects new file automatically
- Databricks job triggered instantly
- Raw file landed in ADLS Gen2 Raw Layer
- Audit log started(file name and timestamp recorded)

TRANSFORM — Databricks Job

- Clean and standardize column names and dates
- Remove currency symbols; look up foreign keys
- Calculate row hash to detect changed records
- Apply data quality checks; separate new/changed

LOAD — Incremental Strategy

- **fact_sales:** New records appended; corrections soft deleted
- **SCD Type 2:** Track history for dimensions needed history

Validation Framework

ABC — Load Integrity

- **Audit:** Every load logged with rows and timestamps
- **Balance:** Source row/revenue must equal target
- **Control:** Pipeline halts on mismatch; no partial loads

Data Quality Checks

- Check for nulls, duplicate records and required non-nulls
- Check Referential integrity, ex: Salesperson must exist in dim_salesperson or Reject orphan records
- Date validity, ex: Sale date cannot be in the future Sale date cannot be before 2040

How This Solution Adds Value

Transforming raw data into actionable insights

From Spreadsheets to database

- Single source of truth replacing 6 disconnected Excel and csv files
- Referential integrity enforced
- Data quality enforced

Leadership Gets Answers in Seconds

- No more manual data reconciliation
- SQL queries replace Excel pivots
- Reliable numbers every time

Built to Scale

- New models, vendors, or salespeople just add rows
- Service contracts ready when launched
- Schema grows with the business

Future Ready Platform

- Start with Azure SQL today; migrate to Synapse later
- Event driven pipelines eliminate manual updates
- Dimensional model supports future analytics